

Documento de Referencia: **Entidad Relación**

Proyecto: Aplicación **"No Te Cortes"**

Sector: **Servicios de Estética e Imagen Personal (CNAE 9602)**

1. Justificación del Modelo Conceptual

El presente Diagrama Entidad-Relación representa la estructura lógica de la información del sistema **"No Te Cortes"**. El diseño se centra en la **Cita** como entidad transaccional núcleo, vinculando a los clientes con el catálogo de servicios. Aunque la implementación física se realiza en **Cloud Firestore (NoSQL)**, este modelo conceptual garantiza la integridad referencial y define las reglas de negocio necesarias para la escalabilidad y seguridad de la agenda en tiempo real.

2. Definición de Entidades y Atributos

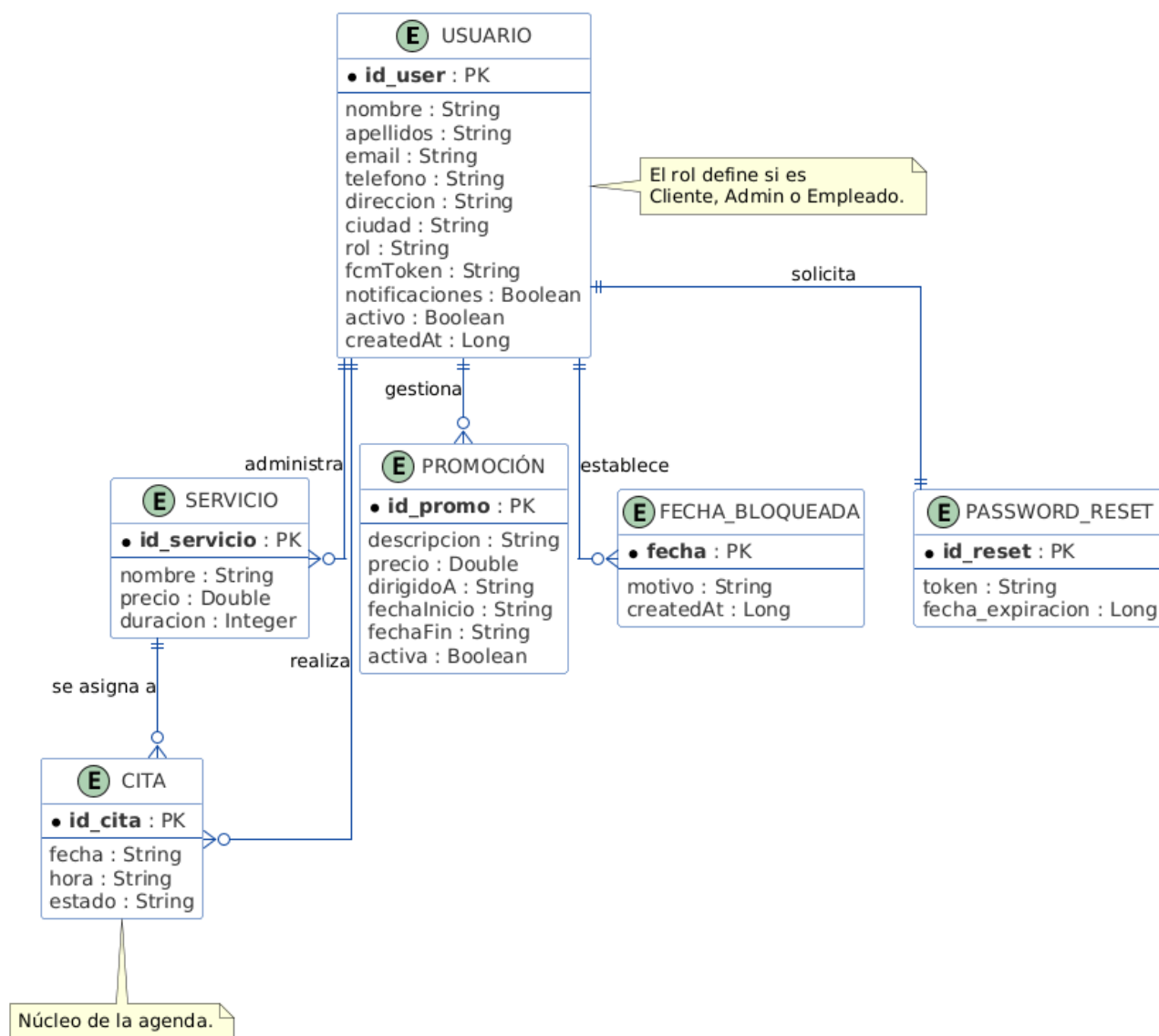
Se han definido las entidades maestras eliminando redundancias técnicas para mantener un enfoque puramente conceptual.

Entidad	Atributos (PK = Clave Primaria)	Descripción
USUARIO	id_user , nombre, apellidos, email, teléfono, dirección, ciudad, fcmToken, notificaciones (bool), activo (bool), createdAt	Usuario registrado. El atributo rol (Admin/Cliente/Empleado) define los permisos de acceso.
SERVICIO	id_servicio , nombre, precio, duración	Catálogo de servicios (Corte, Barba, etc.) gestionado por el administrador.
CITA	id_cita , fecha, hora, estado	Entidad central que formaliza la reserva de un servicio por parte de un usuario.
PROMOCIÓN	id_promo , descripción, activa (bool), precio, dirigidoA, fechaInicio, fechaFin	Ofertas comerciales aplicables al sistema.
FECHA_BLOQUEADA	fecha , motivo, createdAt	Días o franjas no laborables que restringen la creación de citas.
PASSWORD_RESET	id_reset , token, fecha_expiracion	Entidad de soporte para la seguridad y recuperación de

Entidad	Atributos (PK = Clave Primaria)	Descripción
		credenciales.

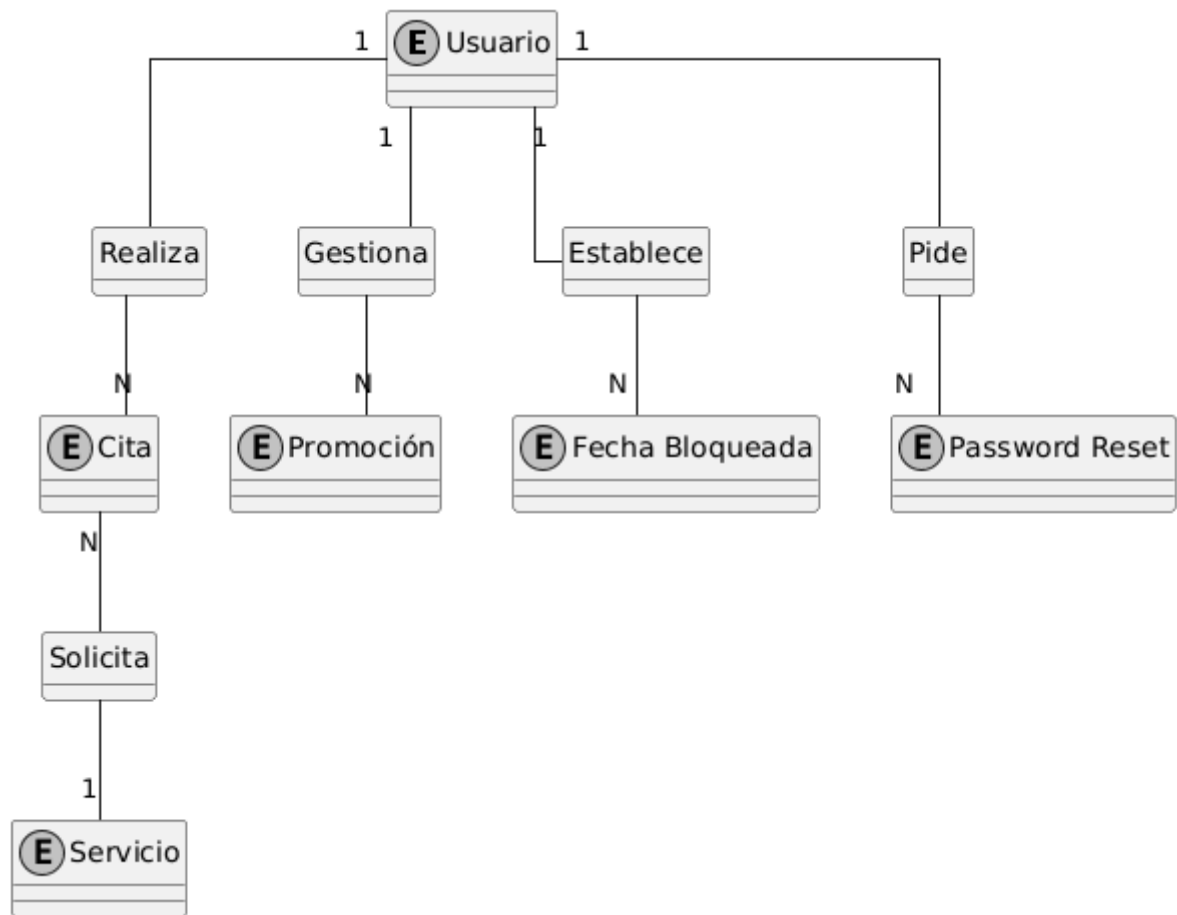
3. Diagrama Entidad-Relación

3.1 Modelo Conceptual (Notación Chen)



Nota técnica: Este diagrama define las reglas de negocio y las interrelaciones mediante una abstracción de alto nivel, cumpliendo con los estándares de modelado conceptual.

3.2 Modelo Lógico / Esquema de Datos



Nota técnica: Representación de la estructura física final. Define las claves primarias (PK) y el tipado de datos necesario para la implementación en el SDK de Firebase.

4. Interrelaciones y Cardinalidades

A continuación, se describen las relaciones lógicas y la justificación de su peso numérico en el sistema, asegurando la integridad referencial de la base de datos:

- USUARIO – (Realiza) – CITA (1:N): Un usuario (cliente) puede registrar y tener asociadas múltiples citas a lo largo del tiempo. La cardinalidad es (1,N) porque un registro en la entidad USUARIO puede estar relacionado con 0 a N registros en la entidad CITA. Por el contrario, cada CITA pertenece de forma obligatoria a un único usuario para garantizar la trazabilidad de la reserva.
- CITA – (Solicita) – SERVICIO (N:1): Muchas citas pueden solicitar el mismo servicio del catálogo (ej. "Degradado"). Se define como (N,1) ya que cada CITA se vincula a

un único servicio para heredar de forma consistente su precio y duración estimada, evitando ambigüedades en la agenda.

- USUARIO (Admin) – (Gestiona) – PROMOCIÓN / SERVICIO (1:N): Representa la lógica de gestión administrativa. Un administrador controla múltiples entradas del catálogo y promociones activas. Solo los perfiles con privilegios de administración (`isAdmin: true`) pueden crear o modificar estos registros.
- USUARIO (Admin) – (Establece) – FECHA_BLOQUEADA (1:N): Un administrador define N fechas de cierre o vacaciones. Cada registro de bloqueo es único y actúa como un filtro de validación que impide la creación de citas en días no laborables.
- USUARIO – (Solicita) – PASSWORD_RESET (1:N): Cada proceso de recuperación de contraseña se vincula al usuario que lo solicita. Aunque el proceso es único por sesión para garantizar la seguridad, un usuario puede generar múltiples solicitudes a lo largo del tiempo si fuera necesario.

5. Validación de Reglas de Negocio y Rendimiento

Para asegurar que este modelo conceptual sea eficiente en un entorno NoSQL, se han integrado las siguientes validaciones:

- **Integridad mediante Security Rules:** Se aplican reglas a nivel de servidor que validan que solo el dueño del `id_user` pueda escribir en sus instancias de CITA.
- **Optimización por Índices Compuestos:** Se han configurado índices (estado + fecha + nombre) para permitir consultas instantáneas sobre la entidad central, evitando cuellos de botella en la visualización de la agenda.
- **Desnormalización Controlada:** Para el historial del cliente, se utiliza la técnica de subcolecciones, permitiendo que la relación \$1:N\$ entre Usuario y Cita se resuelva con latencia mínima.

Nota final: Este modelo garantiza que la aplicación maneje la información de forma segura, sentando las bases para una implementación escalable en el ecosistema Android/Firebase.